

Cross-Model Collaboration for Enhancing LLM-Based Code Generation

JIANGPING HUANG and WENGUANG YE, School of Computer Science and Technology, Chongqing University of Posts and Telecommunications, China

WEISONG SUN*, YEBO FENG, and YANG LIU, Nanyang Technological University, Singapore

Large language models (LLMs) have significantly advanced automatic code generation, yet most existing approaches rely on single-model inference, making their performance susceptible to model-specific limitations and failure patterns. To address this, we introduce Cross-Model Collaborative Scheduling (CMCS), a framework that enables multiple LLMs to collaboratively generate code. At its core, CMCS combines execution-guided self-correction with a peer-to-peer corrective handover mechanism, transferring unresolved problems between models to leverage their complementary strengths and mitigate the limitations of any single model. Experiments on eight code generation benchmarks show that CMCS achieves the strongest performance among the evaluated comparable-scale approaches with strong parameter efficiency. In particular, two 7B-scale models consistently outperform a much larger 30B-scale model on every benchmark where both are evaluated. When applied to proprietary models, CMCS boosts Pass@1 on APPS to 36.00%, more than three times that of the stronger individual model (11.33%). CMCS also maintains competitive accuracy when only 20% of test cases are available for diagnostic feedback. Ablation, scalability, and cost-effectiveness analyses quantify component contributions and practical deployment trade-offs. These findings indicate that cross-model collaboration offers a practical and parameter-efficient alternative to scaling up monolithic models for code generation.

CCS Concepts: • **Software and its engineering** → **Automatic programming**; *Software testing and debugging*; • **Computing methodologies** → **Cooperation and coordination**.

Additional Key Words and Phrases: Code Generation, Large Language Models, Cross-Model Collaboration, Diagnostic Correction Mechanism

1 Introduction

Automatically generating code that accurately corresponds to user requirements offers numerous advantages, such as shortening the software development cycle [24], improving the quality and consistency of the software [12], and reducing human errors and repetitive tasks [60]. Consequently, automatic code generation has garnered significant attention from both academia and industry due to its potential to reduce development costs. Various methods have been explored for code generation. Over the past decades, several classical code generation techniques have been developed, including deductive synthesis [20, 35], inductive synthesis [10, 42], and neural-guided synthesis [4, 36, 58]. These foundational methods laid important groundwork for program synthesis in

*Corresponding author.

Authors' Contact Information: Jiangping Huang, huangjp@cqupt.edu.cn; Wenguang Ye, ywg2093971044@gmail.com, School of Computer Science and Technology, Chongqing University of Posts and Telecommunications, Chongqing, China; Weisong Sun, weisong.sun@ntu.edu.sg; Yebo Feng, yebo.feng@ntu.edu.sg; Yang Liu, yangliu@ntu.edu.sg, Nanyang Technological University, Singapore.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, or post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/8-ART

<https://doi.org/10.1145/3840382>

Software Engineering (SE), demonstrating that automated techniques can reliably translate formal specifications into executable code and improve development efficiency.

Large Language Models (LLMs) based on the Transformer architecture [45] have recently achieved remarkable progress across a wide range of generation tasks [5, 26, 44]. By pretraining on extensive code corpora such as GitHub repositories, these models acquire rich contextual representations that can be effectively adapted to code generation tasks [40, 48]. To further improve their ability to satisfy user intent—e.g., generating code aligned with natural language specifications or passing test cases—prior work has explored various prompting and reasoning strategies [21, 29, 56]. While these approaches enhance LLM-based code generation, they primarily optimize the internal reasoning of a single model. For complex problems, such as those appearing in programming competitions or professional software engineering interviews, even advanced LLMs can still fail to produce correct solutions. Recent studies therefore incorporate self-repair mechanisms [14, 55], enabling models to iteratively diagnose and correct errors in their outputs using execution feedback.

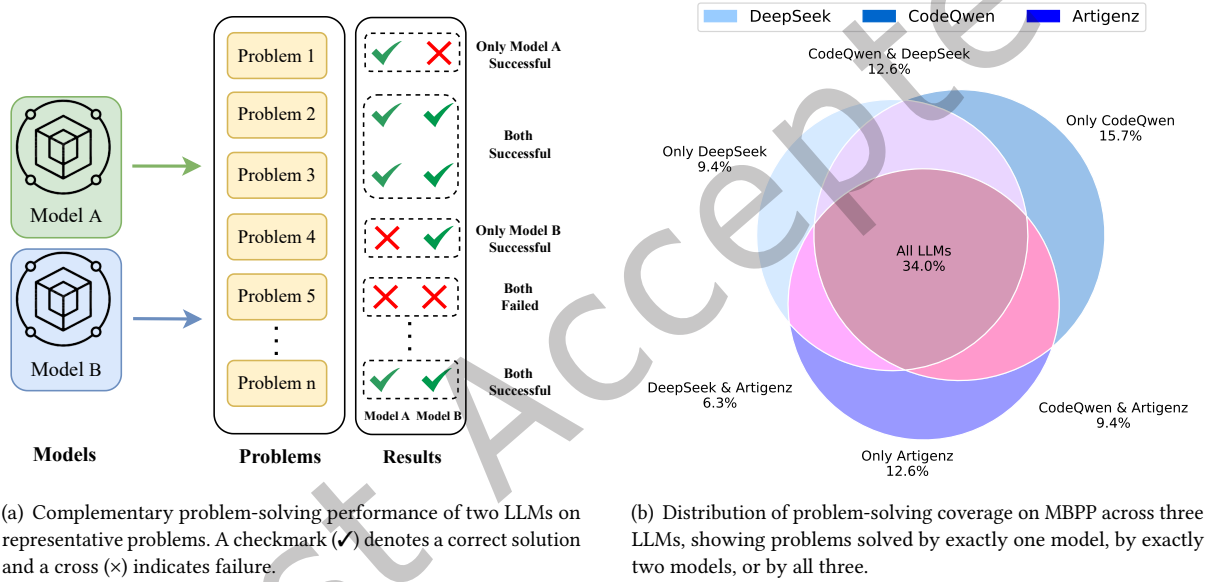


Fig. 1. Illustration of complementary strengths across LLMs. Subfigure (a) shows that different models succeed on different subsets of problems. Subfigure (b) further quantifies this complementarity on MBPP. Percentages in Subfigure (b) are normalized over the problems solved by at least one of the three models.

However, trial-and-error-based self-repair is not always effective, and its iterative execution can incur substantial inference cost. Prior studies show that repeatedly invoking an LLM without corrective feedback or new diagnostic signals often yields minimal improvement, as the model tends to reproduce similar errors across iterations [25, 37, 47, 49]. Moreover, most existing approaches still rely on a single LLM, whose performance is fundamentally constrained by its own biases and knowledge gaps. As illustrated in Fig. 1, different models solve different subsets of problems on MBPP [2], revealing clear complementary strengths rather than a universal best performer. This complementary pattern suggests that enabling cross-model collaboration is a promising direction for improving the overall effectiveness of LLM-based code generation.

In this paper, we propose Cross-Model Collaborative Scheduling (CMCS), a novel and effective framework designed to coordinate collaborative code generation across multiple LLMs. In its current instantiation, CMCS employs two LLMs as the minimal collaborative unit, both for conceptual clarity and practical efficiency. However, the framework itself is not restricted to two models; its scheduling and handover mechanisms are inherently scalable and can be flexibly extended to larger pools of collaborating LLMs. CMCS operates in three phases. First, the problem set is partitioned into two disjoint subsets, and the two participating models independently generate initial code solutions for their assigned subsets in parallel. Each solution then enters a diagnostic correction loop driven by execution feedback and test results. If a model fails to self-correct, CMCS activates its peer-to-peer corrective handover mechanism, assigning the unresolved problem to another model for a fresh attempt at code generation, diagnosis, and repair. This design strategically exploits diverse complementary strengths across models and mitigates the inherent biases of single-model generation, resulting in substantial and consistent improvements in code generation performance.

We conducted comprehensive experiments across eight widely used code generation benchmarks with 7B-scale LLMs, a representative and practically relevant model size for contemporary deployment scenarios. The results show that CMCS consistently improves generation quality by integrating diagnostic correction with collaborative scheduling. Furthermore, our experiments demonstrate that CMCS is model-agnostic: it can integrate LLMs with different architectures and training paradigms as interchangeable collaborators. Our contributions are summarized as follows:

- We present CMCS, a novel cross-model collaboration framework that introduces a peer-to-peer corrective handover mechanism, enabling the exploitation of complementary strengths across LLMs and mitigating the inherent limitations of single-model code generation.
- We introduce a fine-grained diagnostic correction mechanism that guides LLMs through iterative self-repair using compiler, runtime, and test feedback, enabling structured error analysis and emulating the debugging workflow employed by human developers.
- We conduct experiments across eight benchmarks, showing that a dual 7B-scale configuration outperforms a much larger 30B-scale model and validating CMCS's generalizability across backbone models, multi-model scalability, and robustness under incomplete test suites.
- We release the source code to support reproducibility and foster future research on cross-model collaboration for code generation [19].

2 Related Work

2.1 Code Generation with Large Language Models

Accurately understanding user requirements and translating them into correct executable code has long been a central challenge in artificial intelligence (AI)-assisted SE [23, 51]. Traditional code generation approaches [15, 32], largely based on handcrafted templates and rule-based systems, offer limited adaptability to diverse and complex problem specifications.

Recent advances in LLMs, driven by large-scale pretraining on natural language and code corpora, have substantially improved automatic code generation. Modern LLMs, built upon the Transformer architecture [45], now exhibit strong performance across a wide range of programming tasks and have achieved state-of-the-art results on standard code generation benchmarks. Representative open-source models, such as Code Llama [40], WizardCoder [34] and Magicoder [50], along with proprietary systems like GPT-4 [38], continue to push the performance frontier. Moreover, specialized systems such as AlphaCode [31] have demonstrated competitive programming capabilities. Beyond code synthesis, LLMs are increasingly applied to broader SE tasks, including context-aware code completion (e.g., InCoder [11]), automated program repair [30], and test case generation [41].

Despite these advances, most existing approaches still rely on single-model inference, which may suffer from systematic biases, limited reasoning diversity, and constrained error-correction capacity.

2.2 Self-Correction in Code Generation

A key limitation of one-shot generation is the difficulty of producing correct code in a single attempt. Consequently, recent studies increasingly explore enabling LLMs to iteratively refine or repair their outputs—a process often referred to as self-repair or self-correction. This paradigm treats code repair as a post-processing step to improve the reliability of automated code generation [29].

Several strategies have been proposed to facilitate this. For example, Zhang et al. [55] introduced Self-Edit, which uses execution results to guide the model’s corrections. Our work is also inspired by this line of research. Other prominent methods used as baselines in our evaluation include SELF-DEBUGGING [7], which instructs LLMs to explicitly explain the cause of errors before fixing them, and INTERVENOR [46], which simulates a human-like interactive repair process by casting LLMs into “code learner” and “code teacher” roles. Similarly, Self-Planning [27] adds a planning phase to decompose complex problems into sub-problems before generation. These self-correction methods have proven effective but remain constrained by the knowledge and inherent biases of a single model, which may fail to fix errors if the required solution is beyond its reasoning capabilities.

2.3 Collaborative and Multi-Agent Code Generation

To address the inherent limitations of single-model approaches, recent work has increasingly explored collaborative paradigms involving multiple LLMs or specialized agents. These systems coordinate distinct models or prompting strategies to exploit complementary strengths, thereby improving generation robustness and correctness.

Existing studies have explored several cross-model collaboration paradigms, including role-based coordination, multi-model debate/review, cross-model validation, and routing-based collaboration. A common paradigm is role-playing, where agents emulate a software development team. For example, ChatDev [39] constructs a virtual company in which agents such as a CEO, programmer, and tester collaborate through structured dialogues. Extending this line, MetaGPT [17] implements an assembly-line paradigm where agents assume specialized roles (e.g., product manager, architect, engineer) following standard operating procedures. MapCoder [22], one of our key baselines, uses a four-stage pipeline for example retrieval, planning, coding, and debugging. Other approaches emphasize generating diverse solutions from multiple perspectives. Corex [43], for instance, coordinates multi-model collaboration via “Debate” and “Review” paradigms, treating each model as an expert whose insights contribute to a consolidated solution. Some frameworks employ cross-model validation, such as ConTested [9], which uses one LLM to generate tests for another’s code, forming a test-driven refinement loop. AgentCoder [18] similarly establishes a closed-loop system of code generation, test generation, and execution agents. Routing-based approaches dynamically allocate models to subtasks; for instance, MasRouter [53] introduces a dynamic routing mechanism that intelligently selects the most appropriate LLM for different agents and tasks, balancing performance with computational cost.

Our CMCS framework represents a distinct form of cross-model collaboration that shares the goal of exploiting collective intelligence but adopts a different interaction design. Instead of relying on structured role-playing with extensive inter-agent communication or fixed multi-stage pipelines, CMCS employs a symmetric peer-to-peer corrective handover mechanism. Collaboration is activated only when a model fails to self-correct, allowing the partner model to take over unresolved cases. This selective failover strategy reduces unnecessary interactions for straightforward problems and focuses collaboration on more challenging ones, providing a more cost-efficient way to leverage complementary strengths across models.

3 Methodology

3.1 Overview

In this paper, we propose a novel collaborative framework for code generation, as depicted in Fig. 2. Our framework consists of three main components: Dual-Model Collaborative Code Generation (DCCG), Diagnostic Correction Mechanism (DCM), and Cross-Model Collaborative Generation (CMCG). The first component, introduced in §3.2, initializes the process by allowing two LLMs to generate candidate solutions in parallel, improving the efficiency of initial code synthesis. The second component, described in §3.3, enables each model to self-diagnose and repair errors using compiler messages, runtime outputs, and test-case feedback, guided by prompts designed to facilitate logical thought and reasoning. All correction attempts are performed in a stateless manner: each repair query includes the current faulty code and diagnostic feedback without relying on previous conversational history, thereby preventing context accumulation and reasoning drift. The final component, discussed in §3.4, is activated when a model fails to self-correct. In such cases, unresolved problems are conditionally transferred to the collaborating model, enabling the system to exploit complementary reasoning capabilities across LLMs. This selective peer-to-peer corrective handover mechanism focuses cross-model collaboration on challenging cases and improves the robustness of automated code generation.

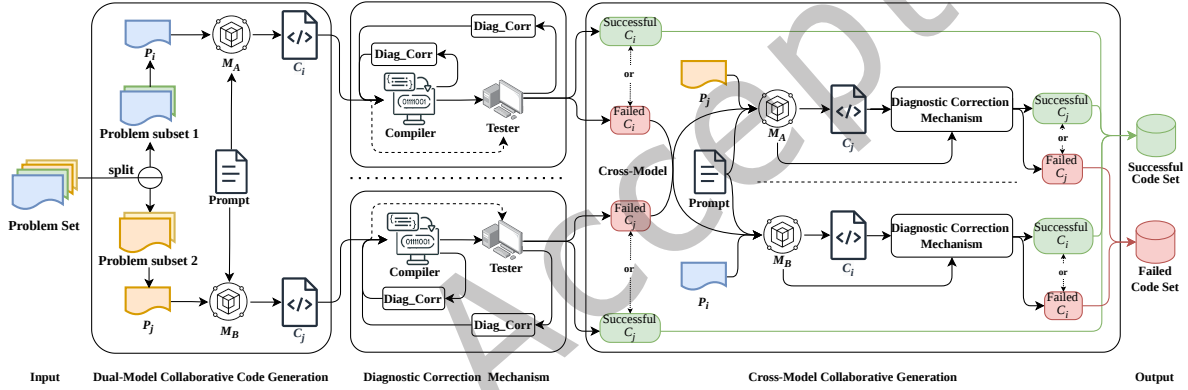


Fig. 2. Overview of the CMCS framework. The problem set consists of natural language specifications that define the target programming tasks. The diagnostic correction mechanism is further illustrated in Fig. 3.

3.2 Dual-Model Collaborative Code Generation

We denote a set of programming problems as $\mathcal{P} = \{P_1, P_2, \dots, P_n\}$, where P_k represents the k -th problem described in natural language. The goal of code generation is to transform each problem $P_k \in \mathcal{P}$ into a corresponding piece of executable code, C_k . Formally, this transformation process can be expressed as:

$$G(P_k) \rightarrow C_k \quad (1)$$

where G denotes the code generation procedure that produces code C_k that is syntactically correct and semantically aligned with the problem description P_k . Note that the natural-language description and the generated program are not required to be aligned in length or structure.

In the context of LLMs, this transformation is guided by a prompt p and executed by a model M . Therefore, Equation (1) can be extended as:

$$G(P_k, p \mid M) \rightarrow C_k \quad (2)$$

where p denotes the designed prompt and M represents the chosen LLM.

Our method utilizes two high-performance LLMs, denoted as M_A and M_B , to collaboratively generate code. To enable concurrent operation, we randomly partition the complete problem set $\mathcal{P}$ into two disjoint subsets of comparable size, $\mathcal{P}_A$ and $\mathcal{P}_B$, which are assigned to M_A and M_B , respectively. This parallel processing strategy eliminates idle time, balances computation across the models, and improves overall generation throughput. Although we focus on the dual-model configuration for clarity and detailed analysis, the framework is naturally extensible to a general set of collaborating models $\mathcal{M} = \{M^{(1)}, M^{(2)}, \dots, M^{(m)}\}$ by partitioning $\mathcal{P}$ into m disjoint subsets and enabling the same failure-triggered collaborative handover mechanism among the models.

To better elicit the ability of LLMs in code generation, we carefully design structured prompt templates. This design is grounded in prior empirical findings [57], which identify “Requirement Misalignment” and “Context Misapplication” as the primary causes of LLM failures in code tasks. Taking the MBPP [2] as an example, our prompt format includes four key sections: **Role**, **Task**, **Requirement**, and **Output**, as shown in **B1**. The **Role** section assigns a specific identity to the LLM (e.g., “a professional coding expert”), providing crucial context. The **Task** section clearly indicates the specific type of *task* to be performed. The **Requirement** section details the specific constraints for code generation, including the test cases (*tests*) that the code must pass. The **Output** section specifies the required format for the LLM’s response, ensuring standardization and facilitating automated parsing of the generated code.

B1: Prompt Templates

Role: You are a professional coding expert.

Task: $\{task\}$.

Requirement:

- The generated code must satisfy the following tests: $\{tests\}$.
- Python code should strictly follow the logical flow and requirements in natural language descriptions.
- Avoid adding output statements that are not mentioned in the description in the code.
- Ensure that all necessary import statements are correctly included, and that all functions and variables are properly defined and declared.
- Code structure should avoid causing compilation or **runtime** errors.

Output: Enclose the provided Python code snippet within triple backticks ``python ``.

3.3 Diagnostic Correction Mechanism

Once the problem subsets $\mathcal{P}_A$ and $\mathcal{P}_B$ are processed by models M_A and M_B to generate initial code solutions, these solutions are subjected to compilation and testing. If a generated code snippet fails validation, we employ DCM to prompt the same LLM to analyze the failure and correct the error. As illustrated in Fig. 3, this mechanism leverages compiler messages, execution outputs, and test feedback to guide iterative self-repair. For languages such as C++, the validation process involves compilation followed by execution against a predefined test suite. Following prior work [55], errors encountered during validation can be categorized into three types:

- **Compilation Errors:** Errors that occur when the code does not conform to syntax rules or language specifications, resulting in the program failing to compile.
- **Runtime Errors:** The program compiles but fails during execution (e.g., timeouts, crashes, or unhandled exceptions), resulting in incomplete test execution.
- **Test Errors:** Errors where the program runs to completion but produces an output that does not match the expected result for a given test case.

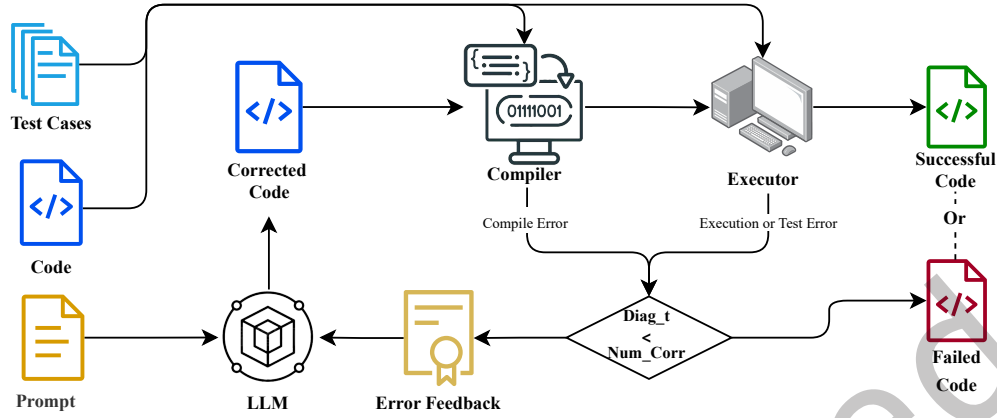


Fig. 3. Illustration of the DCM. The generated code is compiled, executed, and tested. Errors occurring at any stage are diagnosed and corrected by LLMs until the code passes all tests or exceeds the maximum number of correction attempts. **Note:** compilation applies only to compiled languages.

The process of obtaining feedback from compilation, execution, and testing can be formalized as follows:

$$\begin{cases} \mathcal{F}_{Comp}(C) \rightarrow E_{Comp} \\ \mathcal{F}_{Exec}(C) \rightarrow E_{Exec} \\ \mathcal{F}_{Test}(C, T) \rightarrow E_{Test} \end{cases} \quad (3)$$

where $\mathcal{F}_{Comp}$, $\mathcal{F}_{Exec}$, and $\mathcal{F}_{Test}$ denote the procedures that collect error information from the generated code C during the compilation, execution, and testing phases, respectively; T denotes the test suite, and the resulting error signals are denoted as E_{Comp} , E_{Exec} , and E_{Test} . For interpreted languages such as Python, which do not involve a separate compilation phase, the compilation error check is skipped and the mechanism proceeds directly to execution and testing.

B2: Diagnostic Correction Templates

Compilation Error Diagnostic Correction: A compilation error occurred with the following compiler message: $\{error_message\}$. Please diagnose the reasons for these errors by thinking step by step, then refine the generated code: $\{generated_code\}$. You may refer to the requirements: $\{problem\}$.

Execution Error Diagnostic Correction: Although the program has been generated, the following errors occurred during execution: $\{error_message\}$. Please diagnose the reasons for these errors step by step, then refine the generated code: $\{generated_code\}$. You may refer to the requirements: $\{problem\}$.

Test Error Diagnostic Correction: The provided test cases are: $\{input\}$, and the expected output should be: $\{expected_output\}$, but the actual output is: $\{error_message\}$. Please diagnose the reasons for these errors by thinking step by step, then refine the generated code: $\{generated_code\}$. You can refer to the requirements: $\{problem\}$.

We design structured templates to help LLMs interpret three types of error information and reason about how to refine the generated code. These templates are general-purpose and can incorporate additional contextual signals when necessary. For compilation errors, we include the full compiler message, the affected line number, and the surrounding code context. For execution errors, we provide metadata indicating abnormal behavior (e.g., failure to execute, timeout, or unexpected termination). For test errors, where the program completes execution but produces incorrect outputs, we wrap the corresponding inputs, expected outputs, and actual outputs in the designed template. These structured error descriptions offer concrete and interpretable signals for LLMs, enabling more precise diagnostic reasoning during self-correction. The designed template in our implementation is shown in the following text box **B2**.

The diagnostic correction templates are applicable to the compilation, execution, and testing stages, each with a stage-specific template structure. Once instantiated, the corresponding diagnostic feedback is injected into the **Task** section of the prompt (see **B1**). By tracking errors encountered across these three stages and incorporating structured diagnostic signals, an LLM can iteratively refine the generated code. For collaboratively scheduled dual LLMs, we apply the DCM to each model instance. At each stage, a corresponding stage-specific diagnostic correction loop is executed, which can be formalized as follows:

$$C' \leftarrow \text{Diag_Corr}(E_s, C, P, p \mid M), \quad s \in \{\text{Comp, Exec, Test}\}. \quad (4)$$

where s indexes the diagnostic stage, E_s denotes the error information collected at stage s , C is the current code, P is the problem specification, p denotes the instantiated diagnostic prompt, and M represents the LLM instance performing diagnostic correction. The function $\text{Diag_Corr}(\cdot)$ implements the diagnostic correction process, which takes an erroneous code C as input and returns a refined version C' along with its execution status, as detailed in Algorithm 1.

3.4 Cross-Model Collaborative Generation

If a code snippet generated by one model (e.g., M_A) fails to pass the diagnostic correction process, CMCS invokes the CMCG module. This module addresses these self-correction failures by leveraging a collaborating LLM (e.g., M_B). The core idea is simple yet powerful: reassign the failed problem. For instance, the problem that M_A could not solve is now assigned to M_B for a fresh attempt at generation and correction. This approach is motivated by empirical observation that a model may reach its reasoning boundary or fall into repetitive error patterns for certain problems, thus benefiting from a different model's perspective. This method fully utilizes the complementary capabilities and diverse performance preferences of the models. The process is described more formally as follows:

$$G(P', p \mid M') = \begin{cases} G(P_A, p \mid M_B), & \text{if } M_A \text{ fails to correct the code for } P_A, \\ G(P_B, p \mid M_A), & \text{if } M_B \text{ fails to correct the code for } P_B. \end{cases} \quad (5)$$

where P_A and P_B originate from the initial subsets $\mathcal{P}_A$ and $\mathcal{P}_B$, respectively. Here, P' denotes the problem after handover and is defined as $P' \in \{P_A, P_B\}$, and M' denotes the collaborating model selected to process the handover case, where $M' \in \{M_A, M_B\}$. The above formulation describes the dual-model case ($m = 2$). For a general set of collaborating models, the handover can be naturally extended as a sequential failover process. When a model $M^{(i)}$ fails, another model $M^{(j)}$ is selected from the remaining models, i.e., $M^{(j)} \in \mathcal{M} \setminus \mathcal{M}_{\text{used}}$, where $\mathcal{M}_{\text{used}}$ denotes the set of models that have already been used for this problem. The process continues until either a model successfully generates a correct solution or all models in $\mathcal{M}$ have been exhausted. While the current implementation adopts uniform random routing for simplicity and fairness, more informed routing strategies based on model specialization or historical performance may further accelerate convergence and improve computational efficiency, which we leave for future work.

Algorithm 1 Diagnostic Correction Mechanism**INPUT:**

C code to be diagnosed and corrected
 M model used to correct code
 T test case for the problem
 n_{dc} maximum number of diagnostic corrections
 P problem to be solved
 p diagnostic correction prompt

OUTPUT:

C' corrected code
 s status indicating whether C' is correct

```

1: function DIAG_CORR( $C, M, T, n_{dc}, P, p$ )
2:    $C_{current} \leftarrow C$ 
3:   for  $d_i = 1$  to  $n_{dc}$  do
4:      $s, e, E_{Comp} \leftarrow$  compile  $C_{current}$ 
5:     if  $s == \text{"CompilationError"}$  then
6:        $C_{current} \leftarrow$  correct  $C_{current}$  errors with  $M$  based on  $E_{Comp}, P, p$ 
7:     else
8:        $s, E_{Exec}, E_{Test} \leftarrow$  execute  $e$ , test with  $T$ 
9:       if  $s == \text{"RuntimeError"}$  then
10:         $C_{current} \leftarrow$  correct  $C_{current}$  errors with  $M$  based on  $E_{Exec}, P, p$ 
11:       else if  $s == \text{"TestError"}$  then
12:         $C_{current} \leftarrow$  correct  $C_{current}$  errors with  $M$  based on  $E_{Test}, P, p$ 
13:       end if
14:     end if
15:     if  $s == \text{"Success"}$  then
16:       break
17:     end if
18:   end for
19:    $C' \leftarrow C_{current}$ 
20:   return ( $C', s$ )
21: end function

```

▷ Loop for diagnostic corrections
 ▷ Compile (C++) or Syntax Check (Python)
 ▷ Check for compilation errors
 ▷ Correct compilation errors
 ▷ Execute and test code
 ▷ Check for runtime errors
 ▷ Correct runtime errors
 ▷ Check for test errors
 ▷ Correct test errors
 ▷ If code is correct, exit early
 ▷ Return corrected code and its final status

In the CMCG stage, the system receives the outputs from the previous phase, namely the generated code segments C_A and C_B (from models M_A and M_B , respectively), along with their corresponding success statuses s_A and s_B . Based on these outcomes, the system adopts one of the following strategies, as detailed in Algorithm 2.

- **Case 1:** If s_A is success and s_B is failure, the system accepts C_A and delegates the failed problem from M_B to M_A for further correction.
- **Case 2:** If s_A is failure and s_B is success, the same procedure applies with roles reversed: the system accepts C_B and assigns the failed problem from M_A to M_B .
- **Case 3:** If both s_A and s_B are failures, the system initiates parallel CMCG, where each model attempts to correct the other's failed problem.
- **Case 4:** If both s_A and s_B are successes, no further action is taken, and both correct solutions are returned as final outputs.

This cross-model collaboration strategy, enabled by a complementary mechanism, allows the system to adaptively determine appropriate generation pathways based on different test outcomes, thereby improving both

Algorithm 2 Cross-Model Collaborative Generation**INPUT:**

P_A, C_A, s_A	Problem, generated code, and status from Model M_A
P_B, C_B, s_B	Problem, generated code, and status from Model M_B
M_A, M_B	The two collaborating LLMs
T_A, T_B	Test suites for problems P_A and P_B
n_{dc}	Maximum number of diagnostic corrections
p	Prompt for generation/correction

OUTPUT:

(C'_A, s'_A)	Final corrected code and status for problem P_A
(C'_B, s'_B)	Final corrected code and status for problem P_B

```

1: if  $s_A \neq \text{"Success"}$  and  $s_B \neq \text{"Success"}$  then
2:   // Parallel Execution: Swap problems between models
3:    $C_A^* \leftarrow G(P_A, p \mid M_B)$ 
4:    $C_B^* \leftarrow G(P_B, p \mid M_A)$ 
5:    $(C'_A, s'_A) \leftarrow \text{DIAG\_CORR}(C_A^*, M_B, T_A, n_{dc}, P_A, p)$ 
6:    $(C'_B, s'_B) \leftarrow \text{DIAG\_CORR}(C_B^*, M_A, T_B, n_{dc}, P_B, p)$ 
7:   else if  $s_A == \text{"Success"}$  and  $s_B \neq \text{"Success"}$  then
8:      $(C'_A, s'_A) \leftarrow (C_A, s_A)$ 
9:     // Cross-Model Correction for  $C_B$ 
10:     $C_B^* \leftarrow G(P_B, p \mid M_A)$ 
11:     $(C'_B, s'_B) \leftarrow \text{DIAG\_CORR}(C_B^*, M_A, T_B, n_{dc}, P_B, p)$ 
12:    else if  $s_A \neq \text{"Success"}$  and  $s_B == \text{"Success"}$  then
13:       $(C'_B, s'_B) \leftarrow (C_B, s_B)$ 
14:      // Cross-Model Correction for  $C_A$ 
15:       $C_A^* \leftarrow G(P_A, p \mid M_B)$ 
16:       $(C'_A, s'_A) \leftarrow \text{DIAG\_CORR}(C_A^*, M_B, T_A, n_{dc}, P_A, p)$ 
17:    else
18:       $(C'_A, s'_A) \leftarrow (C_A, s_A)$ 
19:       $(C'_B, s'_B) \leftarrow (C_B, s_B)$ 
20:    end if
21:  return  $(C'_A, s'_A), (C'_B, s'_B)$ 

```

▷ Case 3: Both codes failed
 ▷ Generate initial C_A using M_B
 ▷ Generate initial C_B using M_A
 ▷ Correct C_A using M_B
 ▷ Correct C_B using M_A
 ▷ Case 1: Only C_A is correct
 ▷ Keep successful C_A
 ▷ Generate initial C_B using M_A
 ▷ Correct C_B using M_A
 ▷ Case 2: Only C_B is correct
 ▷ Keep successful C_B
 ▷ Generate initial C_A using M_B
 ▷ Correct C_A using M_B
 ▷ Case 4: Both codes are correct

accuracy and efficiency in automated code generation. When one LLM struggles with a problem or converges to repetitive errors, the system reassigns the failed case to the other LLM for further diagnosis and correction. This design ensures that even if a single model misdiagnoses or underperforms, cross-model collaboration can substantially enhance the overall robustness and success rate of CMCS. The complete procedure is described in Algorithm 2.

4 Evaluation

In this section, we introduce a set of research questions (RQs) that guide the empirical study of CMCS, and we describe the corresponding experimental setup, including datasets, baselines, metrics, and implementation details. We conduct comprehensive experiments to investigate these RQs and to rigorously assess both the effectiveness and practical utility of CMCS.

4.1 Research Questions

We evaluate CMCS by defining the following research questions (RQs):

- **RQ1: How does CMCS compare to state-of-the-art methods and simple ensembles?** RQ1 evaluates CMCS against diverse baselines, including direct generation, prompting strategies, and advanced agent-based frameworks. We further compare CMCS with a naïve ensemble to verify the benefit of its collaborative interaction mechanism.
- **RQ2: How well does CMCS generalize across different programming languages and model scales?** RQ2 assesses the versatility of CMCS by evaluating its performance across Python, C++, and Java, and by testing applicability under different model configurations, including collaborations among larger-scale open-source (e.g., 14B) and proprietary models.
- **RQ3: What explains the effectiveness of CMCS?** RQ3 investigates the internal factors contributing to CMCS performance. We conduct ablation studies to quantify the contribution of each component and analyze the necessity of model diversity, determining whether cross-model collaboration yields tangible benefits over single-model self-correction.
- **RQ4: How does CMCS scale with varying numbers of collaborating models and repair iterations?** RQ4 examines the scalability of CMCS by evaluating performance as the number of collaborating models increases and by analyzing efficiency trends under different numbers of diagnostic correction rounds.
- **RQ5: Is CMCS practical with respect to robustness and cost-efficiency?** RQ5 evaluates real-world applicability by measuring robustness under incomplete test suites (i.e., imperfect feedback) and by performing a comprehensive cost-effectiveness analysis that compares latency, memory usage, and token consumption against state-of-the-art approaches.

4.2 Experiment Settings

4.2.1 Datasets. We evaluate CMCS on eight widely used benchmark datasets covering multiple programming languages, including Python, C++, and Java: **HumanEval** [6], **HumanEval+** [33], **HumanEval-X** [59], **MBPP** [2], **MBPP+** [33], **CodeContests** [31], **APPS** [16], and **SPoC** [28].

Among these, HumanEval and MBPP are two widely used benchmark datasets for evaluating code generation models. HumanEval focuses on Python function synthesis tasks, while MBPP includes a series of basic programming tasks along with their corresponding test cases. To further enhance the comprehensiveness of evaluation, researchers have proposed enhanced versions, HumanEval+ and MBPP+, which significantly improve the robustness of evaluation by expanding the coverage of test cases. Additionally, HumanEval-X, as a multilingual extension of HumanEval, is specifically designed to test the multilingual code generation capabilities of models.

For more complex programming challenges, CodeContests simulates competitive programming scenarios, providing more challenging tasks. APPS is a large-scale, multi-difficulty-level collection of programming problems, suitable for comprehensive evaluation of code generation models. For APPS, we follow Islam et al. [22] and use a subset of 150 randomly selected test problems from APPS for evaluation. SPoC contains C++ programs with associated pseudocode and test cases, each representing solutions to competitive programming tasks. The dataset is organized into different subsets for training, evaluation, and testing, known as Train-Train (TRAINTR), Train-Evaluation (TRAINE), and Train-Test (TRAINT) sets. In addition, two held-out test sets are provided: the TESTP set is divided by problems, while the TESTW set is divided by workers. We followed the default data split in the official SPoC¹, and report results on the TESTP and TESTW sets.

For benchmark datasets lacking publicly available test cases, we followed the approach of Zhang et al. [54] by treating the first private test case as a public test case. For MBPP+, we adopted the original three private test cases from MBPP before they were expanded and made public.

¹<https://sumith1896.github.io/spoc/>

4.2.2 Baselines. To our knowledge, existing studies have not explored a peer-to-peer corrective handover mechanism between collaborating LLMs for code generation. To systematically evaluate CMCS, we compare it against a diverse set of baselines, including representative state-of-the-art methods. A brief overview of these approaches is provided below.

- **Direct** [6] uses the original problem descriptions as input for prompting LLMs to generate code, relying solely on the inherent capabilities of the models.
- **Chain-of-Thought (CoT)** [49] guides LLMs to generate intermediate natural language reasoning steps before producing the final solution. We adopt the classic CoT instruction “Let’s think step by step” to elicit such reasoning from the models.
- **Self-Planning** [27] introduces a planning phase into the code generation process by decomposing complex problems into a sequence of solvable sub-problems and formulating corresponding solution steps. This helps the model better understand task intent and reduces the overall difficulty of problem solving.
- **SELF-DEBUGGING** [7] enables LLMs to iteratively improve their own outputs by executing the generated code and using runtime feedback to guide corrective refinement, thereby improving the accuracy of code generation.
- **INTERVENOR** [46] simulates the human interactive code repair process by using LLMs to play the roles of code learner and code teacher, producing a Chain-of-Repair that improves the accuracy of code generation and repair.
- **MapCoder** [22] employs four agents to simulate the program synthesis workflow of human developers, including example retrieval, planning, code generation, and debugging, to improve the accuracy of code generation.
- **Naïve Ensemble:** To isolate the contribution of our collaborative mechanism, we introduce a naïve ensemble baseline. It independently generates code with two different models and marks a problem as solved if either model succeeds. This baseline enables us to separate CMCS’s collaborative diagnostic process from a simple parallel aggregation.

4.2.3 Metrics. For SPoC, we use the metrics from the paper [28] to evaluate the effectiveness of pseudocode-to-code generation using CMCS. This metric primarily focuses on the functional correctness of the code, referring to the proportion of programs with correct functionality among all generated programs. The calculation formula is defined as follows:

$$Acc = \frac{number_{correctCode}}{number_{totalCode}} \quad (6)$$

where $number_{correctCode}$ represents the number of programs that successfully pass the corresponding test cases, and $number_{totalCode}$ represents the total number of programs in the given dataset. In our experiments, we allocate a total diagnostic correction budget of up to 10 attempts for each task.

For the remaining datasets, we adopt the Pass@K metric [6, 52] to evaluate performance, where functional correctness is determined using test cases. For each problem, we generate K code solutions, and a problem is considered solved if any of the generated solutions pass all benchmark test cases. Accordingly, Pass@K denotes the percentage of solved problems. In our experiments, we set $K = 1$, corresponding to a single-sample evaluation. Compared to multi-sample Pass@K settings, Pass@1 better reflects practical development scenarios, where developers typically rely on one-shot generation rather than selecting from multiple candidate solutions.

4.2.4 Implementation details. To ensure a comprehensive evaluation across different model scales and capabilities, we selected a diverse set of LLMs, ranging from parameter-efficient 7B-scale models to larger open-source models and proprietary state-of-the-art systems.

Open-Source Models. To validate the effectiveness and scalability of CMCS across model sizes, we utilized the following models:

- **7B Series:** We selected **CodeQwen1.5-7B-Chat** [3], **DeepSeek-Coder-6.7b-instruct** [13], **Artigenz-Coder-DS-6.7B²**, **CodeLlama-7b-Instruct** [40], and **Nxcode-CQ-7B-orpo³**.
- **14B & 32B & 33B Series:** To explore performance on larger scales, we included **Qwen2.5-Coder-14B-Instruct⁴**, **NextCoder-14B** [1]⁵, **Qwen2.5-Coder-32B-Instruct⁶**, and **DeepSeek-Coder-33B-instruct** [13]. Qwen2.5-Coder-32B-Instruct was evaluated via its official API using the same prompts and decoding settings as our local experiments. Results for DeepSeek-Coder-33B-instruct were obtained from its official repository.

Proprietary Models. To compare CMCS with representative closed-source LLMs, we evaluate **Qwen-Plus-2025-01-25⁷**, **Gemini-2.0-Flash⁸**, **Gemini-2.5-Flash⁹**, and **GPT-4o-2024-08-06¹⁰** via their official APIs or publicly available leaderboards.

In the following sections, CodeQwen1.5-7B-Chat, DeepSeek-Coder-6.7b-instruct, Artigenz-Coder-DS-6.7B, CodeLlama-7b-Instruct, and Nxcode-CQ-7B-orpo will be referred to as CodeQwen, DeepSeek, Artigenz, CodeLlama, and NxCode, respectively.

All experiments involving 7B and 14B parameter models were conducted on a high-performance server equipped with two NVIDIA Quadro RTX 8000 GPUs (48GB VRAM each). To balance memory usage while maintaining generation quality, all models employed for CMCS, the Naïve Ensemble, and the re-implemented baselines were executed in FP16 (half-precision) format. For the *Direct* baseline, we report standard results derived either from official benchmarks or reproduced using full-precision (FP32) to establish a robust reference standard. This configuration facilitates a fair comparison between our FP16-based implementation and a full-precision (FP32) reference setting.

During code generation, we adopted a greedy decoding strategy with temperature set to 0 to ensure deterministic and reproducible results. To avoid truncation, we set `max_new_tokens=4096`, adjusting this value as needed based on dataset-specific requirements. Generation was performed with `num_return_sequences=1`, and terminated using the standard `eos_token_id`.

To support the reproducibility and fairness of our comparative analysis, we adopted a rigorous baseline implementation strategy. For methods with publicly available official repositories, namely INTERVENOR and MapCoder, we used their official source code and adapted it to our evaluation pipeline while preserving their core logic. For prompting-based methods without official implementations (i.e., CoT, Self-Planning, and SELF-DEBUGGING), we implemented them strictly according to the algorithmic descriptions and prompt templates provided in the original papers.

In addition, we aligned the maximum number of correction/refinement iterations across methods. For CMCS, each model was allowed up to $r = 5$ diagnostic correction attempts per task. Therefore, a task reaching the cross-model handover stage could receive at most 10 diagnostic correction attempts across the two collaborating models, excluding the initial generation and the fresh generation performed after handover. For other iterative code generation methods, we set the maximum number of iterations r_z to 10, allowing up to 10 rounds of generation

²<https://artigenz.github.io/artigenz/>

³<https://huggingface.co/NTQAI/Nxcode-CQ-7B-orpo>

⁴<https://huggingface.co/Qwen/Qwen2.5-Coder-14B-Instruct>

⁵<https://huggingface.co/microsoft/NextCoder-14B>

⁶<https://huggingface.co/Qwen/Qwen2.5-Coder-32B-Instruct>

⁷<https://openrouter.ai/qwen/qwen-plus-2025-01-25>

⁸<https://ai.google.dev/gemini-api/docs/models/gemini-2.0-flash>

⁹<https://ai.google.dev/gemini-api/docs/models/gemini-2.5-flash>

¹⁰<https://developers.openai.com/api/docs/models/gpt-4o?snapshot=gpt-4o-2024-08-06>

Table 1. Comparison of Pass@1 (%) on code generation benchmarks. The last row reports the relative improvement of CMCS over the Direct baseline (left value) and over the best-performing baseline for each benchmark (right value). The symbol $\uparrow$ indicates improvement.

Models	Approaches	HumanEval	HumanEval+	MBPP	MBPP+	CodeContests	APPS
CodeQwen	Direct	83.50 $\uparrow$	78.70 $\uparrow$	79.00 $\uparrow$	67.20 $\uparrow$	2.42	3.33
	CoT	76.22	73.78	75.00	79.10	0.61	2.67
	Self-Planning	63.41	57.92	67.31	69.58	2.42	0.67
	SELF-DEBUGGING	84.76	82.93	89.00	88.10	3.03	5.33
	INTERVENOR	85.98	76.22	78.60	80.16	0.61	5.33
	MapCoder	89.02	82.93	78.60	80.16	1.82	4.67
DeepSeek	Direct	78.60 $\uparrow$	70.73 $\uparrow$	72.00 $\uparrow$	76.00 $\uparrow$	3.03	2.67
	CoT	78.05	69.51	73.00	78.04	3.03	3.33
	Self-Planning	68.90	61.59	68.09	74.34	3.03	2.00
	SELF-DEBUGGING	82.32	76.83	81.00	84.13	4.85	4.67
	INTERVENOR	80.49	74.39	74.32	80.16	4.24	4.00
	MapCoder	79.88	76.83	66.54	73.54	4.24	4.67
CodeQwen	Naïve Ensemble	80.49	70.12	76.65	75.66	3.03	5.33
+DeepSeek	CMCS (Ours)	93.29	89.02	91.83	91.53	6.06	11.33
RelImp/RelImp_{Best} $\uparrow$		15.21/4.80$\uparrow$	19.49/7.34$\uparrow$	21.89/3.18$\uparrow$	28.32/3.89$\uparrow$	125.21/24.95$\uparrow$	282.29/112.57$\uparrow$

$\uparrow$ Results sourced from official model reports (see details in Section 4.2.4).

or refinement. This design controls the maximum iterative correction/refinement budget across methods, while CMCS additionally performs a fresh generation only when cross-model handover is triggered. All methods evaluated within our experimental pipeline used the same problem descriptions and the same set of public test cases, ensuring a consistent evaluation setting.

All generated code was executed and evaluated in standardized runtime environments to ensure consistent external feedback across methods. Python programs were run using Python 3.10. For multilingual evaluation on HumanEval-X, C++ solutions were compiled with GCC (Ubuntu 9.4.0-1ubuntu1 20.04.2) 9.4.0 under the `-std=c++17` and `-O2` flags, while Java solutions were compiled and executed using OpenJDK 11.0.27. All evaluations were conducted within a sandboxed Ubuntu 20.04 environment to guarantee consistency and security across programming languages.

Based on a deduplicated inventory of experimental runs and the recorded per-task runtimes, we estimate that the experiments involving locally deployed open-source models required approximately 1,200–1,400 GPU-hours in total on the aforementioned hardware. Runs whose results were reused across multiple research questions or tables were counted only once. Experiments involving proprietary models accessed through APIs were excluded from this estimate because their underlying hardware allocation and compute time were not observable.

4.3 RQ1: Performance Evaluation of CMCS

We evaluate the code generation performance of CMCS, with the results summarized in Table 1. Results for the **Direct** baseline (marked with $\uparrow$) are obtained from official sources. Across all evaluated benchmarks, CMCS consistently outperforms the compared baselines, yielding substantial improvements in code generation accuracy. Notably, these gains are achieved under the same maximum iterative correction/refinement budget of 10 rounds as the iterative baselines such as SELF-DEBUGGING and MapCoder, while CMCS additionally performs a fresh generation only when cross-model handover is triggered.

To quantify performance gains, we report two types of relative improvement. Specifically, Eq. 7 measures the average relative improvement of CMCS over Direct generation across different models, while Eq. 8 reports the relative improvement over the strongest baseline for each benchmark.

$$\text{RelImp} = \frac{1}{|\mathcal{M}|} \sum_{M \in \mathcal{M}} \left(\frac{P_{\text{CMCS}} - P_M^{\text{Direct}}}{P_M^{\text{Direct}}} \right), \quad (7)$$

where $\mathcal{M}$ denotes the set of models (e.g., CodeQwen and DeepSeek), P_M^{Direct} is the Pass@1 of model M under the Direct baseline, and P_{CMCS} is the performance of CMCS on that benchmark.

$$\text{RelImp}_{\text{Best}} = \frac{P_{\text{CMCS}} - \max_{b \in \mathcal{B}} P_b}{\max_{b \in \mathcal{B}} P_b}, \quad (8)$$

where $\mathcal{B}$ denotes the set of baseline methods evaluated on the same benchmark, P_b denotes the Pass@1 of baseline b , and $\max_{b \in \mathcal{B}} P_b$ represents the best-performing baseline result.

Table 1 compares the performance of CMCS with diverse baselines, including individual methods and a Naïve Ensemble, across code generation benchmarks. CMCS consistently outperforms all baselines. For example, on CodeContests, CMCS achieves a Pass@1 of 6.06%, twice the Naïve Ensemble’s 3.03%. This substantial improvement indicates that the gains stem from the diagnostic and cross-model correction mechanisms, rather than simply taking the union of two independently generated model outputs.

In benchmark tasks (e.g., HumanEval and MBPP), CMCS performs particularly well. For example, on HumanEval, CMCS combined with CodeQwen and DeepSeek achieves a Pass@1 accuracy of 93.29%, representing a 15.21% average relative improvement over the Direct baselines. CMCS also achieves 91.53% on MBPP+, with notable improvements over other methods. On more complex tasks, such as APPS and CodeContests, the performance gains of CMCS become even more pronounced. Specifically, on APPS, CMCS achieves a Pass@1 accuracy of 11.33%, corresponding to a 282.29% average relative improvement over the Direct baselines. Moreover, on CodeContests, CMCS attains an average relative improvement of 125.21% over the Direct baselines.

Traditional prompting-based approaches (e.g., CoT and Self-Planning) underperform in certain settings, in some cases yielding lower accuracy than direct code generation. This behavior may be attributed to error accumulation during intermediate reasoning steps, a phenomenon that is particularly evident in smaller-scale open-source LLMs. Overall, most methods achieve relatively strong performance on standard benchmarks such as HumanEval and MBPP. However, performance degrades substantially on more difficult benchmarks, including CodeContests and APPS, where all methods exhibit a notable drop in accuracy. These results highlight the limitations of existing code generation techniques when handling complex programming tasks and underscore the need for more robust collaborative mechanisms.

4.3.1 Comparison with Larger and Proprietary Models. To further examine how CMCS compares with larger single models, we analyze its performance against several representative open-weight and proprietary LLMs. For models with publicly reported standardized benchmark results (e.g., DeepSeek-Coder-33B-instruct and GPT-4o-2024-08-06), we directly report their official leaderboard or repository-reported results to ensure consistency with widely adopted evaluation settings. For proprietary models without standardized public reports under identical configurations, we conduct evaluations using the same prompts, iteration budgets, and decoding settings as in CMCS.

As shown in Table 2, CMCS—combining two 7B-scale models—achieves performance that is competitive with, and in several benchmarks exceeds, substantially larger open-weight models. For example, on HumanEval,

¹¹<https://evalplus.github.io/leaderboard.html>

¹²<https://github.com/deepseek-ai/deepseek-coder>

Table 2. Performance Comparison of CMCS against Proprietary and Larger Models (Pass@1 %)

Model (Parameters)	HumanEval	HumanEval+	MBPP	MBPP+	APPS	CodeContests
DeepSeek-Coder-33B-instruct	79.30	75.00	70.00	65.60	-	-
Qwen2.5-Coder-32B-Instruct	90.85	85.37	88.33	89.42	8.00	4.85
Qwen-Plus-2025-01-25	90.85	86.59	87.55	91.27	8.00	9.09
GPT-4o-2024-08-06	92.70	87.20	87.60	72.20	-	-
Gemini-2.0-Flash	89.02	83.54	86.38	92.59	11.33	41.82
Gemini-2.5-Flash	96.34	93.29	94.94	97.62	17.33	50.91
CMCS (Ours)	93.29	89.02	91.83	91.53	11.33	6.06

Note: Results for GPT-4o-2024-08-06 are obtained from the EvalPlus leaderboard¹¹, and results for DeepSeek-Coder-33B-instruct are sourced from its official repository¹². All reported numbers correspond to the same benchmark versions as used in our evaluation. “-” indicates that the original source does not report results on the corresponding benchmark.

CMCS achieves 93.29%, compared to 79.30% for DeepSeek-Coder-33B-instruct and 90.85% for Qwen2.5-Coder-32B-Instruct. On MBPP, CMCS (91.83%) also compares favorably with these 30B-scale models. On APPS, CMCS (11.33%) surpasses Qwen2.5-Coder-32B-Instruct (8.00%), and performs on par with Gemini-2.0-Flash (11.33%). These results highlight the parameter efficiency of cross-model collaboration, suggesting that performance gains are not solely attributable to increased model scale. Instead, well-designed collaboration can match or even exceed the performance of substantially larger single-model systems under certain benchmark settings.

When compared with proprietary frontier systems, CMCS remains competitive on standard function-level benchmarks. For instance, CMCS achieves 93.29% on HumanEval and 91.83% on MBPP, surpassing GPT-4o-2024-08-06, Qwen-Plus-2025-01-25, and Gemini-2.0-Flash on these benchmarks. On additional benchmarks, including HumanEval+, MBPP+, and APPS, CMCS maintains competitive performance, demonstrating its robustness across multiple benchmarks. Meanwhile, frontier-scale models such as Gemini-2.5-Flash achieve the strongest performance across all evaluated benchmarks, including both function-level tasks and more complex algorithmic reasoning benchmarks (e.g., CodeContests), reflecting the continued strength of large-scale frontier models.

Overall, these findings suggest that while scaling remains a powerful driver of absolute performance [8], CMCS provides a parameter-efficient and practically deployable alternative. It narrows the performance gap between 7B-scale open-source models and substantially larger proprietary systems, without requiring additional model scaling. As frontier single models continue to improve, extending the proposed collaborative framework to frontier-scale models constitutes a natural and meaningful direction for further investigation.

Answer to RQ1: CMCS demonstrates strong and consistent performance across all evaluated benchmarks. It not only outperforms individual baselines and prompting-based methods, but also surpasses a naïve ensemble approach, suggesting the effectiveness of its interactive collaboration mechanism. Moreover, CMCS exhibits notable parameter efficiency: the CMCS configuration combining CodeQwen and DeepSeek (7B and 6.7B) consistently outperforms substantially larger 30B-scale models across these benchmarks. These results indicate that CMCS provides a parameter-efficient alternative paradigm to scaling up a single model for high-performance code generation.

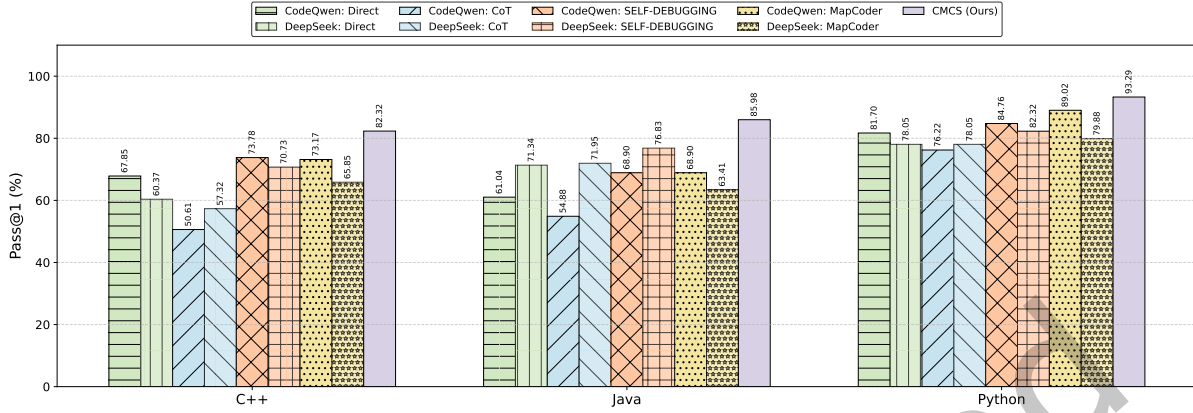


Fig. 4. Performance comparison of CMCS against representative single-model baseline methods on the HumanEval-X benchmark across three programming languages: C++, Java, and Python.

4.4 RQ2: Generalization Capabilities of CMCS

To evaluate the generalization capability of CMCS, we examine its performance across two critical dimensions: programming languages and model scales.

4.4.1 Performance Across Different Programming Languages. We conducted a cross-language evaluation using HumanEval-X, covering three widely-used languages: C++, Java, and Python. The experimental results are shown in Fig. 4. Overall, CMCS demonstrates consistent proficiency across all tested languages, substantially outperforming baseline methods.

Specifically, on the **C++** benchmark, CMCS achieves a Pass@1 score of 82.32%. It consistently outperforms the best-performing baselines under this setting, including SELF-DEBUGGING (73.78%) and MapCoder with CodeQwen (73.17%). Moreover, compared to DeepSeek with SELF-DEBUGGING (70.73%), CMCS yields an improvement of 11.59 percentage points. On the **Java** benchmark, CMCS achieves the highest Pass@1 score of 85.98%. It outperforms all baselines, including DeepSeek with SELF-DEBUGGING (76.83%) and CodeQwen with SELF-DEBUGGING (68.90%), resulting in a 9.15 percentage point improvement over the best-performing baseline. CMCS performs strongly on the **Python** benchmark, achieving a Pass@1 score of 93.29%. This result outperforms all compared baselines in our evaluation, exceeding CodeQwen’s MapCoder (89.02%) by 4.27 percentage points.

4.4.2 Generalization Across Model Scales and Types. To validate the generalizability of CMCS beyond the 7B scale, we investigated its performance when applying the framework to larger open-source models (14B) as well as proprietary models. We conducted experiments in three representative settings, with results summarized in Table 3.

- **Collaboration with 14B Models:** When pairing two 14B models (Qwen2.5-Coder-14B-Instruct and NextCoder-14B), CMCS achieves a Pass@1 of 95.73% on HumanEval+ and a remarkable 22.67% on the challenging APPS benchmark, corresponding to a more than threefold improvement over individual 14B models on APPS and demonstrating that the benefits of collaborative scheduling generalize to larger model scales.
- **Heterogeneous Collaboration (14B+7B):** Pairing a 14B model with a 7B model also yields strong performance, achieving a Pass@1 of 21.33% on APPS, which demonstrates the flexibility of CMCS in effectively coordinating heterogeneous models.

Table 3. Generalization Performance of CMCS across Different Model Sizes and Model Types (Pass@1 %).

Scenario	Method/Model	HumanEval+	MBPP+	APPS
14B + 14B	Direct (Qwen2.5-Coder-14B-Instruct)	81.10	88.89	6.00
	Direct (NextCoder-14B)	87.20	87.57	7.33
	CMCS (Qwen2.5-Coder-14B-Instruct + NextCoder-14B)	95.73	96.56	22.67
14B + 7B	Direct (NextCoder-14B)	87.20	87.57	7.33
	Direct (CodeQwen)	78.70	67.20	3.33
	CMCS (NextCoder-14B + CodeQwen)	93.29	96.30	21.33
Proprietary	Direct (Gemini-2.0-Flash)	83.54	92.59	11.33
	Direct (Qwen-Plus-2025-01-25)	86.59	91.27	8.00
	CMCS (Gemini-2.0-Flash + Qwen-Plus-2025-01-25)	95.12	95.77	36.00

- **Collaboration with Proprietary Models:** Further evidence is observed when pairing two high-performing proprietary models (Gemini-2.0-Flash and Qwen-Plus-2025-01-25). Their collaboration under CMCS increases the APPS Pass@1 score to an outstanding 36.00%, substantially outperforming the better-performing single model.

Answer to RQ2: CMCS demonstrates strong generalization across two key dimensions. First, it achieves the highest performance among the evaluated methods across multiple programming languages (Python, C++, and Java), consistently outperforming competitive baselines such as MapCoder. Second, the framework scales effectively to larger model configurations: collaborations involving 14B and proprietary models yield substantial gains, particularly on challenging benchmarks such as APPS. These results support the model-agnostic and scalable nature of CMCS.

4.5 RQ3: Mechanism and Rationale

In this section, we investigate the underlying mechanisms driving CMCS’s effectiveness. We analyze the complementary behavior of different LLMs, the role of model diversity in comparison to simple ensembles, and the contribution of individual framework components.

4.5.1 The Value of Model Complementarity. To understand why collaboration works, we first analyze individual model strengths and then evaluate pairwise combinations among four representative models (CodeQwen, DeepSeek, Artigenz, and CodeLlama) on 500 MBPP samples.

Fig. 5 illustrates this complementarity. The results show that, for each model pair, a substantial fraction of problems is solved by only one model, while another fraction is solved independently by both models. For instance, in the Artigenz and DeepSeek combination, 47.4% of problems were solved by both, but a crucial 10.8% were solved only by Artigenz and 13.0% only by DeepSeek. This suggests that no single LLM is universally superior; rather, each exhibits distinct strengths and blind spots. This phenomenon underscores the potential for a collaborative framework to overcome individual model limitations.

This complementary nature translates into consistent quantitative performance, as shown in Fig. 6, which reports results for pairs formed from CodeQwen, DeepSeek, Artigenz, and NxCode. Different pairwise collaborations consistently achieve high Pass@1 scores on HumanEval, further validating the model-agnostic nature of

CMCS. The framework flexibly leverages diverse model combinations, rather than relying on a specific model choice, to achieve high performance.

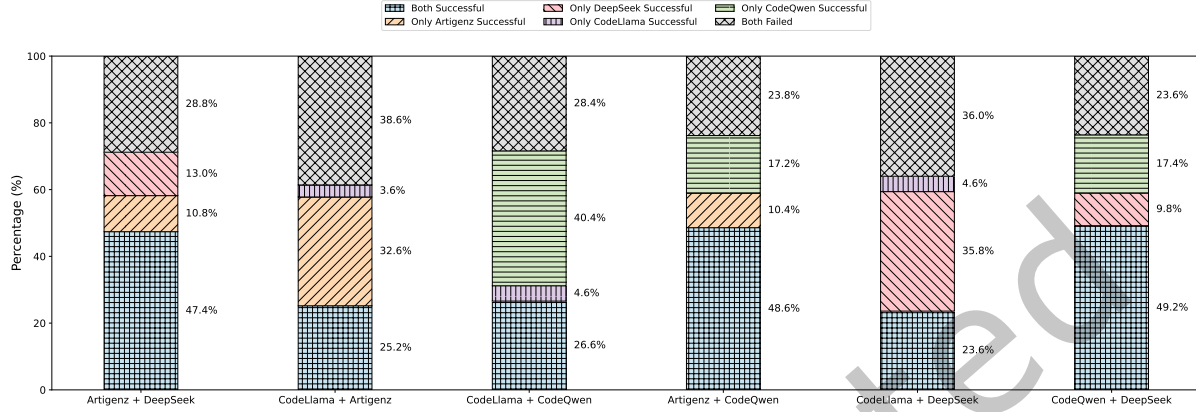


Fig. 5. The distribution of problem-solving outcomes across different model combinations on MBPP. “Both Successful” denotes cases where both models independently produce correct solutions. “Only Model M Successful” indicates cases where only one model succeeds. “Both Failed” represents problems that neither model can solve independently, highlighting the potential space for collaborative improvement.

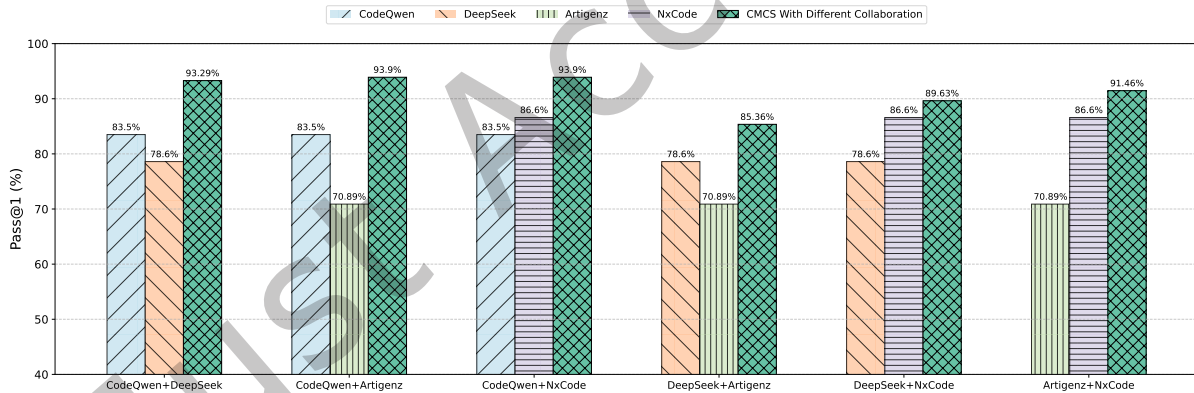


Fig. 6. Pass@1 performance of CMCS with different pairwise model combinations on HumanEval, demonstrating robust performance across heterogeneous model pairings.

4.5.2 The Necessity of Model Diversity. While we have shown that models are complementary, a critical question remains: is this diversity *necessary* for CMCS’s success, or does the improvement stem from the collaborative framework itself rather than model diversity? To answer this question, we conducted a controlled comparison between cross-model and same-model collaboration.

The results, presented in Table 4, provide clear evidence that cross-model collaboration consistently outperforms same-model configurations across all four benchmarks. The ‘CodeQwen+DeepSeek’ pairing surpasses both

‘CodeQwen+CodeQwen’ and ‘DeepSeek+DeepSeek’. On MBPP, for example, the cross-model setup achieves a Pass@1 score of 91.83%, representing an improvement of over 10 percentage points compared to the average of the two same-model setups. These findings support our core hypothesis that model diversity is a key driver of CMCS’s effectiveness. The performance gains are not merely an ensemble effect arising from increased inference attempts; rather, they stem from heterogeneous reasoning patterns and complementary error-correction behaviors that expand the effective solution space and enable the framework to address more challenging problems.

Table 4. Comparison between cross-model and same-model collaboration within CMCS (Pass@1 %).

Model Combination	HumanEval	HumanEval+	MBPP	MBPP+
CMCS(CodeQwen+CodeQwen)	92.68	86.59	75.09	75.93
CMCS(DeepSeek+DeepSeek)	85.36	80.49	80.93	77.25
CMCS(CodeQwen+DeepSeek)	93.29	89.02	91.83	91.53

4.5.3 Ablation Study. To rigorously evaluate the contribution of each component and the necessity of our dual-model architecture, we conducted an extended ablation study as shown in Table 5.

Impact of Dual-Model Architecture (Replacing DCCG with CoT). To evaluate the contribution of the dual-model architecture, we replaced DCCG with a standard single-model CoT baseline (Row 1). Adding DCM to this single-model baseline (Row 2) improves HumanEval performance from 76.22% to 89.63%, but it still underperforms the full CMCS framework (93.29%). This result indicates that diagnostic correction within a single model is insufficient to match the performance of the complete framework. The additional gains arise from combining diagnostic correction with cross-model generation and corrective handover.

Impact of DCM (Missing DCM). We further investigated the “Missing DCM” scenario, where the diagnostic correction loop is removed, leaving only generation and cross-model swapping, referred to as CMCG (Row 3). In this setting, CMCS reduces to a simple cross-model generation and swapping mechanism, achieving 80.49% on HumanEval. The sharp decline from the full model (93.29%) to the w/o DCM variant indicates that cross-model collaboration alone is insufficient to sustain high accuracy without the fine-grained guidance of the diagnostic repair process.

Synergistic Effects. Comparing the complete CMCS framework (Row 5) against the variants lacking cross-model collaboration (w/o CMCG, Row 4) or correction (w/o DCM, Row 3), we observe that CMCS achieves the highest Pass@1 accuracy on both benchmarks. Specifically on MBPP, the full CMCS framework reaches 91.83%, outperforming the single-model correction variant (Row 2) and the Naïve Ensemble, which corresponds to the w/o DCM setting (Row 3). These results confirm that the three components—DCCG, DCM, and CMCG—are not merely additive but synergistic, working together to maximize code generation reliability by jointly reinforcing both generation diversity and error-correction effectiveness.

4.5.4 Qualitative Example Analysis. To further illustrate the synergy suggested by the ablation study, we present a qualitative analysis. An example problem and corresponding outputs using CMCS prompting are shown in Fig. 7. The workflow operates as follows: Initially, the system generates code using a single LLM and applies a diagnostic correction mechanism. If the single model fails to correct the errors (as seen in the intermediate step), CMCS activates the cross-model mechanism. By incorporating a second model with a complementary reasoning perspective, the persistent error is resolved. This example illustrates how CMCS leverages diverse model strengths to mitigate the limitations inherent in any single LLM.

Table 5. Ablation study of CMCS components (DCCG, DCM, and CMCG), analyzing the contribution of each component to overall code generation performance.

Settings	DCCG	DCM	CMCG	HumanEval (%)	MBPP (%)
<i>Single-Model Baselines (Replacing DCCG with CoT)</i>					
1. CoT Baseline	×	×	×	76.22	75.00
2. CoT + DCM (Single-Model Variant)	×	✓	×	89.63	89.00
<i>Component Ablation within CMCS(CodeQwen+DeepSeek)</i>					
3. w/o DCM (No Repair)	✓	×	✓	80.49	76.65
4. w/o CMCG (No Swap)	✓	✓	×	84.76	80.93
5. CMCS (Full Framework)	✓	✓	✓	93.29	91.83

Note: Rows 1–2 are single-model variants based on CodeQwen. Row 1 uses standard CoT generation, while Row 2 further applies the proposed DCM to the CoT-generated solution. Row 2 is therefore a DCM-based single-model variant rather than the Self-Debugging baseline reported in Table 1. Rows 3–4 ablate individual CMCS components by removing diagnostic correction (DCM) or cross-model collaborative generation (CMCG), respectively, while Row 5 reports the full CMCS framework.

Answer to RQ3: The effectiveness of CMCS can be attributed to two primary mechanisms. First, **model diversity** contributes substantially to performance gains: cross-model collaboration leverages complementary reasoning behaviors and diverse failure tendencies across different LLMs, and consistently outperforms same-model collaboration in our evaluation. Second, **component synergy** further enhances effectiveness: ablation results indicate that the integration of dual-model collaborative generation, diagnostic correction, and peer-to-peer corrective handover yields stronger performance than any individual component or partial configuration.

4.6 RQ4: Scalability of Collaboration

This section explores the performance boundaries of CMCS by scaling the collaboration framework along two dimensions: the number of models and the number of repair iterations.

4.6.1 Scalability Across the Number of Collaborating Models. A key question is whether CMCS can be effectively extended beyond a two-model setup. To investigate this, we scale the collaboration from a dual-model to a four-model configuration while keeping all models at 7B-scale, thereby isolating the effect of adding more collaborating models from that of using larger models.

The results in Table 6 indicate that CMCS scales positively as the number of collaborating models increases. Starting from a single CodeQwen model, sequentially adding DeepSeek (second model), Artigenz (third model), and NxCode (fourth model) yields consistent performance improvements across all evaluated benchmarks (HumanEval, HumanEval+, MBPP, MBPP+). The most pronounced improvement occurs when moving from a

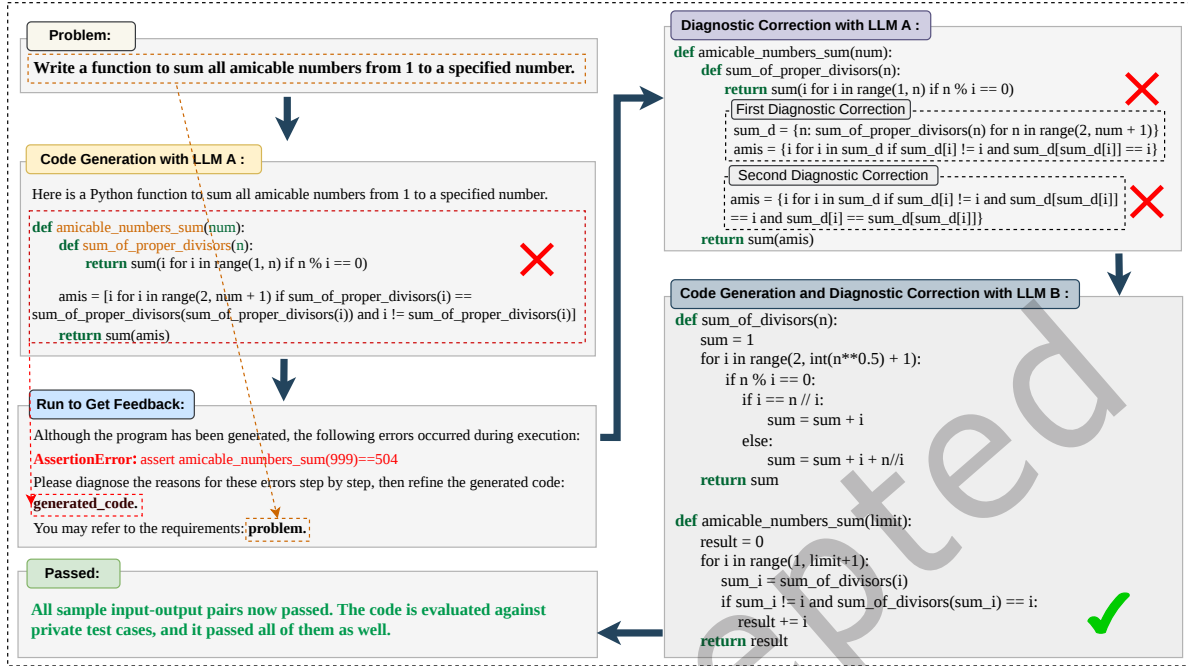


Fig. 7. A qualitative example illustrating how CMCS leverages complementary reasoning across models to resolve persistent errors that cannot be fixed by single-model diagnostic correction.

single model to a dual-model configuration, while further gains are observed as additional models are incorporated. The largest absolute gain is observed on the MBPP+ benchmark, increasing from 67.20% (single CodeQwen) to 92.59% (four-model CMCS). Although the performance of CMCS continues to improve as additional models are incorporated, the magnitude of incremental gains gradually diminishes beyond the dual-model configuration. These findings suggest that CMCS benefits from expanding the pool of diverse collaborators, supporting its applicability beyond pairwise collaboration and highlighting its potential as a general multi-model collaboration framework. This observation also indicates that the dual-model configuration may represent a practical balance between performance improvement and computational overhead.

Table 6. Scalability of CMCS with respect to the number of collaborating models (starting with CodeQwen and sequentially adding DeepSeek, Artigenz, and NxCode) (Pass@1 %).

Combination (Number of Models)	HumanEval	HumanEval+	MBPP	MBPP+
Direct (CodeQwen)	83.50	78.70	79.00	67.20
CMCS (2 Models)	93.29	89.02	91.83	91.53
CMCS (3 Models)	95.12	89.63	93.39	92.33
CMCS (4 Models)	95.73	91.46	94.97	92.59

4.6.2 *Diagnostic Correction Analysis.* Beyond model scaling, we analyzed the scalability of performance relative to the number of diagnostic correction attempts.

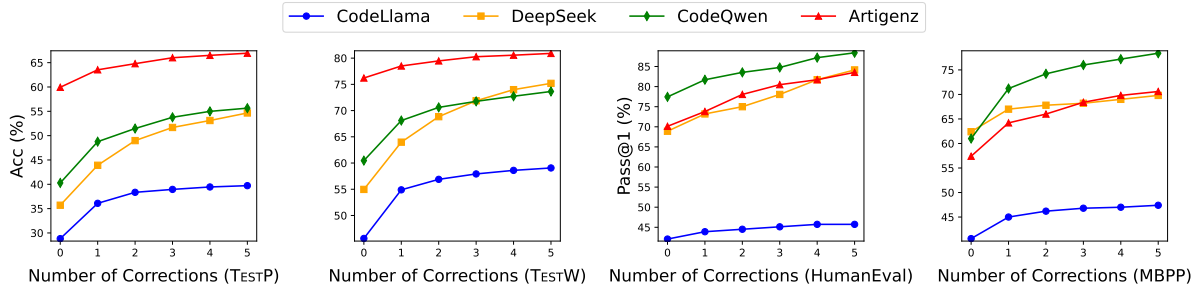


Fig. 8. Performance trends of diagnostic correction across different LLMs on SPoC, HumanEval, and MBPP.

Fig. 8 illustrates the performance trends as the number of correction rounds increases. Across the four evaluation settings (TESTP, TESTW, HumanEval, and MBPP), performance increases by over 10 percentage points on average after five correction rounds. However, the results also exhibit a clear diminishing-returns trend. The largest gains generally occur in the early correction rounds, particularly between the initial state (0 corrections) and the first correction. As the number of corrections increases, the marginal gains tend to decrease, with only relatively small improvements observed in the later rounds.

The results indicate that while the DCM improves accuracy at the cost of additional computation, the performance gains gradually plateau. This underscores the importance of selecting an appropriate correction budget (e.g., five rounds) to balance accuracy improvements against computational efficiency and avoid unnecessary overhead from excessive iterations.

Answer to RQ4: The CMCS framework exhibits strong scalability along two dimensions. Along the collaboration dimension, performance consistently improves as the number of collaborating models increases, with the most pronounced gain observed when moving from a single model to a dual-model configuration. Along the iteration dimension, the DCM effectively enhances accuracy across correction rounds, with the largest improvements occurring in the early stages, demonstrating the efficiency of the iterative design.

4.7 RQ5: Practicality and Cost-Effectiveness

Finally, we investigate the practical applicability of CMCS by systematically evaluating its robustness under incomplete test suites and its computational efficiency.

4.7.1 Robustness to Incomplete Test Oracles. A key concern for real-world deployment is the availability of complete test suites, which are often limited or partially specified in practical software development scenarios. To evaluate CMCS's robustness under more realistic conditions, we conducted an experiment on HumanEval+ in which the diagnostic correction loop was driven by a randomly sampled subset of the test suite, while the final solutions were evaluated against the complete test suite to measure true Pass@1 accuracy. As illustrated in Fig. 9, CMCS exhibits **graceful degradation**: even when only 20% of test cases are available for feedback during correction, the final solutions still achieve 86.59% Pass@1 when evaluated on the full 100% test suite, representing a substantial improvement over the 70.12% Naïve Ensemble baseline (0% feedback) and maintaining strong performance despite severely limited feedback. These results indicate that CMCS remains effective under

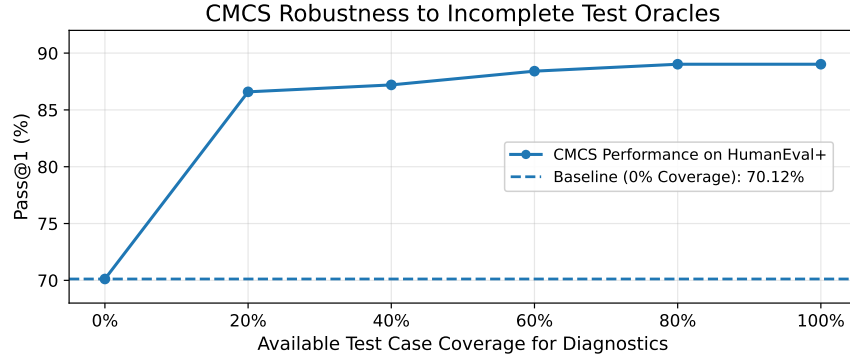


Fig. 9. Robustness of CMCS on HumanEval+ under varying levels of incomplete test oracles, with final solutions evaluated against the complete test suite.

imperfect feedback conditions, suggesting its practical applicability in real-world development environments where complete test oracles are unavailable.

4.7.2 Cost-Effectiveness Analysis. We conduct a cost-effectiveness analysis of CMCS by comparing it with representative baselines under the same hardware setup (two NVIDIA Quadro RTX 8000 GPUs). Table 7 reports Pass@1, average wall-clock time, peak GPU memory usage, and average token consumption on HumanEval and MBPP, enabling a dataset-specific trade-off analysis.

On both HumanEval and MBPP, Direct is the fastest and most resource-efficient baseline, but it generally underperforms the best-performing iterative and collaborative methods in Pass@1. Iterative baselines (e.g., SELF-DEBUGGING and INTERVENOR) improve Pass@1 at the cost of increased runtime and token usage. MapCoder achieves strong accuracy with CodeQwen on HumanEval (89.02%), but incurs the highest computational overhead among the compared methods (e.g., 174.52s and 11,470 tokens). When paired with DeepSeek, MapCoder exhibits notably higher latency on both datasets (329.73s on HumanEval and 418.81s on MBPP), accompanied by consistently high token consumption (11,202–11,840 tokens). This higher overhead is likely related to MapCoder’s multi-stage prompting pipeline and longer generations, which increases inference complexity and resource consumption, particularly for smaller backbone models. Moreover, MapCoder does not consistently outperform other baselines across models and datasets (e.g., 66.54% on MBPP with DeepSeek), indicating variation across backbone models and tasks.

In contrast, CMCS achieves the highest Pass@1 among the compared methods on both datasets while maintaining competitive efficiency. On HumanEval, CMCS reaches 93.29% with 39.55s average runtime, which is about 4.4× faster than MapCoder with CodeQwen (174.52s), while maintaining superior accuracy. On MBPP, CMCS attains 91.83% with 49.04s runtime, about 3.1× faster than MapCoder with CodeQwen (153.79s). Notably, CMCS uses 1,660 tokens on HumanEval and 1,770 tokens on MBPP, which is comparable to or lower than SELF-DEBUGGING on the same datasets, and substantially lower than MapCoder and INTERVENOR. The primary trade-off lies in peak GPU memory usage: CMCS requires concurrently loading two models (e.g., 38.76GB on HumanEval), whereas single-model baselines generally require lower peak memory.

Overall, the results on both datasets suggest that CMCS offers a favorable accuracy–efficiency trade-off: it substantially improves Pass@1 over single-model baselines while maintaining competitive runtime and token usage with increased peak memory from dual-model deployment.

Table 7. Cost-effectiveness comparison of CMCS and baseline methods on HumanEval and MBPP, reporting accuracy, runtime, peak GPU memory usage, and token consumption.

Benchmark	Method / Model	Pass@1(%)	Avg. Time(s)	Peak GPU Memory(GB)	Avg. Tokens
HumanEval	CodeQwen				
	Direct	83.50	18.15	14.08	430
	SELF-DEBUGGING	84.76	68.71	14.40	1,800
	INTERVENOR	85.98	39.04	14.20	3,440
	MapCoder	89.02	174.52	28.12	11,470
	DeepSeek				
	Direct	78.60	25.33	13.06	430
	SELF-DEBUGGING	82.32	64.31	13.52	1,570
	INTERVENOR	80.49	41.49	13.20	4,013
	MapCoder	79.88	329.73	27.74	11,202
	CodeQwen + DeepSeek				
	CMCS (Ours)	93.29	39.55	38.76	1,660
MBPP	CodeQwen				
	Direct	79.00	18.39	14.12	400
	SELF-DEBUGGING	89.00	70.19	14.58	1,980
	INTERVENOR	78.60	37.69	14.24	4,340
	MapCoder	78.60	153.79	17.17	9,290
	DeepSeek				
	Direct	72.00	26.12	13.22	490
	SELF-DEBUGGING	81.00	75.54	13.64	1,510
	INTERVENOR	74.32	52.32	13.86	4,590
	MapCoder	66.54	418.81	25.85	11,840
	CodeQwen + DeepSeek				
	CMCS (Ours)	91.83	49.04	27.92	1,770

Answer to RQ5: CMCS demonstrates strong practical applicability. It maintains robust performance under incomplete test oracles, supporting its use in realistic software development settings with imperfect feedback. Moreover, despite higher peak memory requirements, CMCS achieves the highest Pass@1 among the compared methods while maintaining competitive runtime and token usage, offering a favorable accuracy–efficiency trade-off.

5 Threats to Validity

We discuss potential threats to the validity of our empirical study and the mitigation strategies adopted to reduce their impact, organized into the following validity aspects.

Generalizability of Results. A primary threat concerns the generalizability of our findings. To mitigate this threat, we evaluate CMCS on eight widely used benchmark datasets, including HumanEval, MBPP, and APPS, covering diverse code generation tasks and programming languages. These benchmarks have been widely adopted in prior studies for evaluating code generation performance. The consistent performance of CMCS across these benchmarks suggests that the framework generalizes across these diverse datasets and task settings. Nevertheless, we acknowledge that performance may vary on additional benchmarks or in real-world industrial scenarios, which remain directions for future investigation.

Data Leakage in Pre-trained Models. Another potential threat concerns data leakage, where pre-trained LLMs may have been exposed to benchmark data during training. While such exposure cannot be conclusively verified, this risk applies uniformly to all evaluated methods, as the same underlying backbone models are used for CMCS and all baselines. Although asymmetric exposure across models could theoretically influence collaboration, its precise impact is difficult to quantify. Since our evaluation emphasizes *relative* performance comparisons under identical model configurations, this threat is unlikely to invalidate the comparative conclusions.

Dependency on Test Oracles. A key threat to the practical applicability of CMCS is its reliance on test oracles to provide feedback for the diagnostic correction mechanism. Our primary benchmarks, such as HumanEval and MBPP, provide comprehensive test suites, allowing controlled evaluation. However, in real-world development, complete test suites are often unavailable. To assess this limitation, we conducted a robustness analysis (Section 4.7, Fig. 9) under partial test coverage. The results demonstrate graceful degradation, with CMCS maintaining competitive performance even when only partial feedback is available. Although this mitigates the concern to some extent, further evaluation under alternative feedback mechanisms (e.g., static analysis or automatically generated tests) would strengthen the construct validity of the framework.

Execution Safety and Resource Costs. The execution of machine-generated code may pose safety risks when running unverified programs. To mitigate this concern, all experiments were conducted in a sandboxed environment to isolate execution and prevent system-level side effects. In addition, the collaborative design of CMCS requires loading multiple models simultaneously, resulting in higher peak GPU memory usage compared to single-model approaches. As analyzed in our cost-effectiveness evaluation (Table 7), this increased memory footprint reflects an inherent trade-off between resource consumption and performance. While CMCS incurs higher peak memory usage, it maintains a competitive balance between accuracy and latency. This trade-off should therefore be considered in resource-constrained scenarios.

6 Conclusion

In this paper, we presented CMCS, a collaborative framework for improving LLM-based code generation by leveraging the complementary strengths of heterogeneous models. By integrating parallel generation, diagnostic self-correction, and a peer-to-peer corrective handover mechanism, CMCS alleviates model-specific limitations commonly observed in single-model systems. Extensive experiments across eight diverse benchmark datasets demonstrate that structured cross-model collaboration consistently yields substantial performance gains over competitive baselines. Importantly, CMCS achieves these improvements without increasing the scale of individual models, highlighting that performance advancement in code generation does not solely depend on parameter expansion. This finding suggests that structured collaboration itself provides a viable and scalable pathway for enhancing robustness and reliability in code generation. Building on this insight, future work will explore several promising directions, including extending CMCS to support dynamic routing over larger pools of collaborating models, further optimizing the trade-off between performance and resource consumption in constrained environments, and enhancing real-world applicability by incorporating weaker or alternative feedback signals in place of fully specified test suites. Overall, this work establishes structured cross-model collaboration as a powerful and generalizable paradigm for advancing the reliability and robustness of code-generating LLMs.

Acknowledgments

We thank the anonymous reviewers for their insightful and constructive comments. This work was supported by the Humanities and Social Sciences Fund of the Ministry of Education (Grant No. 20YJCZH047), the Science and Technology Research Program of the Chongqing Municipal Education Commission (Grant No. KJQN202500631), and the Research Project of the Japan Research Center of Chongqing University of Posts and Telecommunications (Grant No. K2020-222).

References

- [1] Tushar Aggarwal, Swayam Singh, Abhijeet Awasthi, Aditya Kanade, and Nagarajan Natarajan. 2025. NextCoder: Robust Adaptation of Code LMs to Diverse Code Edits. In *Proceedings of the 42nd International Conference on Machine Learning* (Proceedings of Machine Learning Research, Vol. 267), Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (Eds.). PMLR, 616–638. <https://proceedings.mlr.press/v267/agarwal25b.html>
- [2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. arXiv:2108.07732 [cs.PL] <https://arxiv.org/abs/2108.07732>
- [3] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. 2023. Qwen Technical Report. arXiv:2309.16609 [cs.CL] <https://arxiv.org/abs/2309.16609>
- [4] Rohan Bavishi, Caroline Lemieux, Roy Fox, Koushik Sen, and Ion Stoica. 2019. AutoPandas: neural-backed generators for program synthesis. *Proc. ACM Program. Lang.* 3, OOPSLA, Article 168 (Oct 2019), 27 pages. doi:10.1145/3360594
- [5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bf8ac142f64a-Paper.pdf
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgan Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] <https://arxiv.org/abs/2107.03374>
- [7] Xinyun Chen, Maxwell Lin, Nathanael Schaerli, and Denny Zhou. 2024. Teaching Large Language Models to Self-Debug. In *International Conference on Learning Representations*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun (Eds.), Vol. 2024. 8746–8825. https://proceedings.iclr.cc/paper_files/paper/2024/file/2460396f2d0d421885997dd1612ac56b-Paper-Conference.pdf
- [8] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2023. PaLM: Scaling Language Modeling with Pathways. *Journal of Machine Learning Research* 24, 240 (2023), 1–113. <http://jmlr.org/papers/v24/22-1144.html>
- [9] Jinhao Dong, Jun Sun, Wenjie Zhang, Jin Song Dong, and Dan Hao. 2025. ConTested: Consistency-Aided Tested Code Generation with LLM. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA027 (June 2025), 22 pages. doi:10.1145/3728902

- [10] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. 2021. DreamCoder: bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (Virtual, Canada) (PLDI 2021). Association for Computing Machinery, New York, NY, USA, 835–850. doi:10.1145/3453483.3454080
- [11] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A Generative Model for Code Infilling and Synthesis. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=hQwb-lbM6EL>
- [12] Pranav Garg and Srinivasan H. Sengamedu. 2022. Synthesizing code quality rules from examples. *Proc. ACM Program. Lang.* 6, OOPSLA2, Article 187 (Oct 2022), 31 pages. doi:10.1145/3563350
- [13] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. arXiv:2401.14196 [cs.SE] <https://arxiv.org/abs/2401.14196>
- [14] Kavi Gupta, Peter Ebert Christensen, Xinyun Chen, and Dawn Song. 2020. Synthesize, Execute and Debug: Learning to Repair for Neural Program Synthesis. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 17685–17695. https://proceedings.neurips.cc/paper_files/paper/2020/file/cd0f74b5955dc87fd0605745c4b49ee8-Paper.pdf
- [15] Tihomir Gvero and Viktor Kuncak. 2015. Interactive Synthesis Using Free-Form Queries. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 2. 689–692. doi:10.1109/ICSE.2015.224
- [16] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring Coding Challenge Competence With APPS. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, J. Vanschoren and S. Yeung (Eds.), Vol. 1. https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/file/c24cd76e1ce41366a4bbe8a49b02a028-Paper-round2.pdf
- [17] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VtmBAGCN7o>
- [18] Dong Huang, Jie M. Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. 2024. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. arXiv:2312.13010 [cs.CL] <https://arxiv.org/abs/2312.13010>
- [19] Jiangping Huang, Wenguang Ye, Weisong Sun, Yebo Feng, and Yang Liu. 2025. CMCS: Official Implementation. <https://github.com/CSMA-Research-Group/CMCS>. Accessed: 2025-12-30.
- [20] Kangjing Huang, Xiaokang Qiu, Peiyuan Shen, and Yanjun Wang. 2020. Reconciling enumerative and deductive program synthesis. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (London, UK) (PLDI 2020). Association for Computing Machinery, New York, NY, USA, 1159–1174. doi:10.1145/3385412.3386027
- [21] Tao Huang, Zhihong Sun, Zhi Jin, Ge Li, and Chen Lyu. 2024. Knowledge-Aware Code Generation with Large Language Models. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension* (Lisbon, Portugal) (ICPC '24). Association for Computing Machinery, New York, NY, USA, 52–63. doi:10.1145/3643916.3644418
- [22] Md. Ashraful Islam, Mohammed Eunus Ali, and Md Rizwan Parvez. 2024. MapCoder: Multi-Agent Code Generation for Competitive Problem Solving. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 4912–4944. doi:10.18653/v1/2024.acl-long.269
- [23] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. 2018. Mapping Language to Code in Programmatic Context. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, Brussels, Belgium, 1643–1652. doi:10.18653/v1/D18-1192
- [24] S. Jacindha, G. Abishek, and P. Vasuki. 2022. Program Synthesis—A Survey. In *Computational Intelligence in Machine Learning*, Amit Kumar, Jacek M. Zurada, Vinit Kumar Gunjan, and Raman Balasubramanian (Eds.). Springer Nature Singapore, Singapore, 409–421. https://doi.org/10.1007/978-981-16-8484-5_39
- [25] Bin Ji, Huijun Liu, Mingzhe Du, and See-Kiong Ng. 2024. Chain-of-Thought Improves Text Generation with Citations in Large Language Models. In *Proceedings of the AACL Conference on Artificial Intelligence*, Vol. 38. 18345–18353. doi:10.1609/aaai.v38i16.29794
- [26] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825 [cs.CL] <https://arxiv.org/abs/2310.06825>
- [27] Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. 2024. Self-Planning Code Generation with Large Language Models. *ACM Trans. Softw. Eng. Methodol.* 33, 7, Article 182 (Sept. 2024), 30 pages. doi:10.1145/3672456

- [28] Sumith Kulal, Panupong Pasupat, Kartik Chandra, Mina Lee, Oded Padon, Alex Aiken, and Percy S Liang. 2019. SPoC: Search-based Pseudocode to Code. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc., https://proceedings.neurips.cc/paper_files/paper/2019/file/7298332f04ac004a0ca44cc69ecf6f6b-Paper.pdf
- [29] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu Hong Hoi. 2022. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 21314–21328. https://proceedings.neurips.cc/paper_files/paper/2022/file/8636419dea1aa9fbd25fc4248e702da4-Paper-Conference.pdf
- [30] Fengjie Li, Jiajun Jiang, Jiajun Sun, and Hongyu Zhang. 2025. Hybrid Automated Program Repair by Combining Large Language Models and Program Analysis. *ACM Trans. Softw. Eng. Methodol.* 34, 7, Article 202 (Aug. 2025), 28 pages. doi:10.1145/3715004
- [31] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustín Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with AlphaCode. *Science* 378, 6624 (2022), 1092–1097. doi:10.1126/science.abq1158
- [32] Greg Little and Robert C. Miller. 2007. Keyword programming in java. In *Proceedings of the 22nd IEEE/ACM International Conference on Automated Software Engineering (Atlanta, Georgia, USA) (ASE '07)*. Association for Computing Machinery, New York, NY, USA, 84–93. doi:10.1145/1321631.1321646
- [33] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 21558–21572. doi:10.52202/075280-0943
- [34] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2024. WizardCoder: Empowering Code Large Language Models with Evol-Instruct. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=UnUwSigK5W>
- [35] Z. Manna and R. Waldinger. 1992. Fundamentals of deductive program synthesis. *IEEE Transactions on Software Engineering* 18, 8 (1992), 674–704. doi:10.1109/32.153379
- [36] Benjamin Mariano, Yanju Chen, Yu Feng, Greg Durrett, and İşıl Dillig. 2022. Automated transpilation of imperative to functional code using neural-guided program synthesis. *Proc. ACM Program. Lang.* 6, OOPSLA1, Article 71 (Apr 2022), 27 pages. doi:10.1145/3527315
- [37] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. 2024. Is Self-Repair a Silver Bullet for Code Generation?. In *International Conference on Learning Representations*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun (Eds.), Vol. 2024. 36545–36593. https://proceedings.iclr.cc/paper_files/paper/2024/file/9ddc141bdf9d1db510cefff56c586ad-Paper-Conference.pdf
- [38] OpenAI, Josh Achiam, Steven Adler, et al. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] <https://arxiv.org/abs/2303.08774>
- [39] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 15174–15186. doi:10.18653/v1/2024.acl-long.810
- [40] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2024. Code Llama: Open Foundation Models for Code. arXiv:2308.12950 [cs.CL] <https://arxiv.org/abs/2308.12950>
- [41] Jiho Shin, Sepehr Hashtroudi, Hadi Hemmati, and Song Wang. 2024. Domain Adaptation for Code Model-Based Unit Test Case Generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (Vienna, Austria) (ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 1211–1222. doi:10.1145/3650212.3680354
- [42] Saurabh Srivastava, Sumit Gulwani, Swarat Chaudhuri, and Jeffrey S. Foster. 2011. Path-based inductive synthesis for program inversion. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (San Jose, California, USA) (PLDI '11)*. Association for Computing Machinery, New York, NY, USA, 492–503. doi:10.1145/1993498.1993557
- [43] Qiushi Sun, Zhangyue Yin, Xiang Li, Zhiyong Wu, Xipeng Qiu, and Lingpeng Kong. 2024. Corex: Pushing the Boundaries of Complex Reasoning through Multi-Model Collaboration. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*. <https://openreview.net/forum?id=cCQRjpOru5>
- [44] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971 [cs.CL] <https://arxiv.org/abs/2302.13971>
- [45] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach,

- R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [46] Hanbin Wang, Zhenghao Liu, Shuo Wang, Ganqu Cui, Ning Ding, Zhiyuan Liu, and Ge Yu. 2024. INTERVENOR: Prompting the Coding Ability of Large Language Models with the Interactive Chain of Repair. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 2081–2107. doi:10.18653/v1/2024.findings-acl.124
- [47] Yuki Wang, Gonzalo Gonzalez-Pumariaga, Yash Sharma, and Sanjiban Choudhury. 2023. Demo2Code: From Summarizing Demonstrations to Synthesizing Code via Extended Chain-of-Thought. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 14848–14956. doi:10.52202/075280-0655
- [48] Yue Wang, Hung Le, Akhilesh Gotmare, Nghi Bui, Junnan Li, and Steven Hoi. 2023. CodeT5+: Open Code Large Language Models for Code Understanding and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 1069–1088. doi:10.18653/v1/2023.emnlp-main.68
- [49] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 24824–24837. https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf
- [50] Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. 2024. Magicoder: Empowering Code Generation with OSS-Instruct. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*, Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.). PMLR, 52632–52657. <https://proceedings.mlr.press/v235/wei24h.html>
- [51] Pengcheng Yin and Graham Neubig. 2018. TRANX: A Transition-based Neural Abstract Syntax Parser for Semantic Parsing and Code Generation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Eduardo Blanco and Wei Lu (Eds.). Association for Computational Linguistics, Brussels, Belgium, 7–12. doi:10.18653/v1/D18-2002
- [52] Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Qianxiang Wang, and Tao Xie. 2024. CoderEval: A Benchmark of Pragmatic Code Generation with Generative Pre-trained Models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal) (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 37, 12 pages. doi:10.1145/3597503.3623316
- [53] Yanwei Yue, Guibin Zhang, Boyang Liu, Guancheng Wan, Kun Wang, Dawei Cheng, and Yiyan Qi. 2025. MasRouter: Learning to Route LLMs for Multi-Agent Systems. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 15549–15572. doi:10.18653/v1/2025.acl-long.757
- [54] Huan Zhang, Wei Cheng, Yuhua Wu, and Wei Hu. 2024. A Pair Programming Framework for Code Generation via Multi-Plan Exploration and Feedback-Driven Refinement. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (Sacramento, CA, USA) (ASE '24)*. Association for Computing Machinery, New York, NY, USA, 1319–1331. doi:10.1145/3691620.3695506
- [55] Kechi Zhang, Zhuo Li, Jia Li, Ge Li, and Zhi Jin. 2023. Self-Edit: Fault-Aware Code Editor for Code Generation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 769–787. doi:10.18653/v1/2023.acl-long.45
- [56] Tianyi Zhang, Tao Yu, Tatsunori Hashimoto, Mike Lewis, Wen-Tau Yih, Daniel Fried, and Sida Wang. 2023. Coder Reviewer Reranking for Code Generation. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 41832–41846. <https://proceedings.mlr.press/v202/zhang23av.html>
- [57] Tanghaoran Zhang, Yue Yu, Xinjun Mao, Shangwen Wang, Kang Yang, Yao Lu, Zhang Zhang, and Yuxin Zhao. 2025. Instruct or Interact? Exploring and Eliciting LLMs' Capability in Code Snippet Adaptation through Prompt Engineering. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (Ottawa, Ontario, Canada) (ICSE '25)*. IEEE Press, 566–577. doi:10.1109/ICSE55347.2025.00104
- [58] Yating Zhang. 2020. Scalability and precision improvement of neural program synthesis. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (Virtual Event, Australia) (ASE '20)*. Association for Computing Machinery, New York, NY, USA, 1391–1393. doi:10.1145/3324884.3418912
- [59] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. 2023. CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Benchmarking on HumanEval-X. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (Long Beach, CA, USA) (KDD '23)*. Association for Computing Machinery, New York, NY, USA, 5673–5684. doi:10.1145/3580305.3599790
- [60] Amit Zohar and Lior Wolf. 2018. Automatic Program Synthesis of Long Programs with a Learned Garbage Collector. In *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett (Eds.), Vol. 31.

Curran Associates, Inc., 2098–2107. https://proceedings.neurips.cc/paper_files/paper/2018/file/390e982518a50e280d8e2b535462ec1f-Paper.pdf

Just Accepted